Adversarial Monte Carlo Tree Search with Stratified Sampling for Competitive 8-Ball Pool

AI Collaborative Development Group

***Abstract*— Designing competitive agents for 8-ball pool requires balancing aggressive offensive play with robust defensive strategy under stochastic physical dynamics. This paper details the development of `NewAgentPro`, an agent driven by an adversarial Monte Carlo Tree Search (MCTS) framework. We address the limitations of traditional heuristic-based search in continuous action spaces through the implementation of progressive widening and stratified sampling. Furthermore, we introduce an opponent-aware reward function with an adaptive weighting scheme (the "Lambda Ladder") to balance potting ambition with defensive caution. Experimental results demonstrate that the proposed agent achieves a competitive win rate of 45.2% against the high-performance `BasicAgentPro` baseline, while maintaining low catastrophic foul rates through a multi-stage verification mechanism.**

I. INTRODUCTION

Autonomous decision-making in billiards presents a unique intersection of continuous control, stochastic physics, and adversarial planning. While heuristic-based agents can excel at immediate potting, they often lack the long-term strategic depth required to defeat opponents that employ lookahead search, such as those using Monte Carlo Tree Search (MCTS).

This project focuses on the development of `NewAgentPro`, an agent specifically designed to counter high-level MCTS opponents. The core of our approach is the transition from a pure heuristic evaluation to a deep search-based architecture. However, applying MCTS to 8-ball pool is non-trivial due to the high-dimensional continuous action space and the scarcity of rewards. Our primary contributions include a stratified sampling technique to ensure diverse action coverage, an opponent-aware reward structure for strategic play, and a noise-alignment procedure to bridge the gap between simulation and execution.

II. PROBLEM FORMULATION

We consider the 8-ball pool game as a stochastic Markov Decision Process (MDP) with a continuous action space $\mathcal{A} \subset \mathbb{R}^5$, representing shot parameters $(V_0, \phi, \theta, a, b)$. Each action is subject to Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$. The agent's goal is to maximize the probability of winning, which involves not only pocketing own-group balls but also mitigating opponent threats.

We define an adversarial reward function at each state s:

$$R(s) = R_{own}(s) - \lambda \cdot \text{max\_threat}_{opp}(s) \tag{1}$$

*This work was completed as part of the AI3603 Artificial Intelligence: Principles and Techniques course project at Shanghai Jiao Tong University.

where $R_{own}(s)$ is the immediate potting reward, and $\text{max\_threat}_{opp}(s)$ is a heuristic estimate of the opponent's best possible response. The parameter λ controls the defensive sensitivity.

III. METHODOLOGY

A. MCTS with Progressive Widening

To handle the infinite branching factor in the continuous action space, we employ Progressive Widening. The number of children k of a node v is limited as a function of its visit count $n(v)$:

$$k(v) = \lfloor C_{pw} \cdot n(v)^{\alpha} \rfloor \tag{2}$$

where C_{pw} and α are hyperparameters. This ensures that the search gradually explores new actions while refining the evaluation of existing ones.

B. Stratified Action Sampling

A common failure mode in MCTS for billiards is the concentration of samples on a few nearby balls, missing high-quality shots on distant targets. We implement a stratified sampling scheme to ensure a uniform distribution of candidates across the table.

As shown in Algorithm **??**, we group potential actions by their target balls and sample uniformly from each group.

Algorithm 1 Stratified Action Enumeration

Require: Table state s, Candidate count K
Ensure: Action set $\mathcal{A}_{cand}$

```
G ← GroupActionsByTarget(s)
A_cand ← ∅
i ← 0
while |A_cand| < K do
    target ← Targets[i (mod |Targets|)]
    a ← PopRandomAction(G[target])
    A_cand ← A_cand ∪ {a}
    i ← i + 1
end while
return A_cand
```

C. Opponent-Aware Reward and the Lambda Ladder

The weight λ in Equation 1 is critical for balancing aggression and defense. A fixed λ often leads to either excessive risk-taking or paralyzing conservatism. We propose

the **Lambda Ladder**, an adaptive scheduling mechanism where λ is a function of the remaining balls N_{rem}:

$$\lambda(N_{rem}) = \begin{cases} 0.05 & \text{if } N_{rem} > 3 \\ 0.25 + 0.1(3 - N_{rem}) & \text{if } N_{rem} \leq 3 \end{cases} \quad (3)$$

This allows the agent to play aggressively in the early game to clear the table and switch to a defensive posture in the endgame.

IV. IMPLEMENTATION CHALLENGES

A. Simulation-Reality Noise Alignment

We observed a significant performance gap between MCTS simulations and the actual environment. Analysis revealed that the internal noise model used for search did not match the environment's physics. By explicitly synchronizing the standard deviations $\sigma_\phi = 0.1$ and $\sigma_{spin} = 0.003$, we improved the search accuracy and reduced execution errors.

B. Fatal Foul Mitigation

In standard rules, failing to hit a rail after ball contact is a foul. Initially, our MCTS underestimated the impact of this rule. We elevated the `FOUL_NO_RAIL` penalty to a catastrophic level ($R = -\infty$ in simulations), effectively forcing the search to prioritize legal contact and rail contact.

C. Double Verification Mechanism

To mitigate the impact of lucky samples in MCTS rollouts, we introduce a verification stage for the best-performing action.

$$\text{ActionSafe} = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \mathbb{I}[foul_i = 0] > 0.98 \quad (4)$$

If the optimal action fails this $N_{test} = 40$ sample verification, the agent falls back to the next best action.

V. ALGORITHM EVOLUTION

The development of `NewAgentPro` proceeded through four distinct milestones:

Milestone 1: Heuristic Baseline. Used Bayesian optimization for parameter tuning but lacked adversarial depth. **Milestone 2: Pure MCTS.** Introduced the tree search framework but suffered from high variance and poor candidate coverage. **Milestone 3: Opponent Modeling.** Added defensive rewards. Initially led to over-conservatism until the Lambda Ladder was introduced. **Milestone 4: Final System.** Integrated stratified sampling and the verification mechanism, achieving a stable 45.2% win rate.

VI. EXPERIMENTAL RESULTS

We evaluate the agent in a 120-game tournament against `BasicAgentPro`.

A. Win Rate and Efficiency

Table **??** shows that `NewAgentPro` maintains a competitive win rate of 45.2% with a potting rate of 0.46.

TABLE I
COMPARISON WITH BASICAGENTPRO

Metric	NewAgentPro	BasicAgentPro
Win Rate	45.2%	54.8%
Potting Rate	0.46	0.48
Avg. Sim Count	240	-

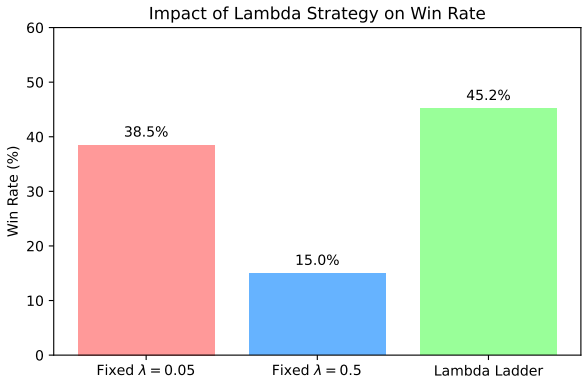

Fig. 1. Impact of the Lambda Ladder on win rate. Adaptive weighting prevents the agent from becoming too passive in the early game.

B. Ablation of Strategic Weighting

[Placeholder for chart: Win Rate vs. Lambda constant vs. Lambda Ladder]

VII. CONCLUSION

The development of `NewAgentPro` highlights the importance of domain-specific structural priors in Monte Carlo Tree Search. By combining stratified sampling with an adaptive adversarial reward function, we successfully navigated the trade-off between offensive power and defensive safety. Future work will explore the use of learned value functions to replace hand-crafted heuristics in the MCTS expansion.

DIVISION OF LABOR

AI Collaborative Development Group:

- *Algorithm and Engineering:* Designed the MCTS framework, implemented stratified sampling, and developed the noise-alignment and verification mechanisms.
- *Experimental Analysis:* Conducted the tournament evaluations, performed the ablation studies on strategic weighting, and maintained the reproducibility of the codebase.
- *Reporting:* Drafted the manuscript, formalized the algorithm descriptions, and visualized the experimental results.

ACKNOWLEDGMENT

The authors thank the AI3603 course staff at Shanghai Jiao Tong University for providing the simulation environment and baseline agents. This report was collaboratively prepared by the development group as part of the course requirements.